

# Python

## Python 速成

### 关于 Python

Python 是一门已在世界上广泛使用的解释型语言。它提供了高效的高级数据结构，还能简单有效地面向对象编程，也可以在算法竞赛。

#### Python 的优点

- Python 是一门 **解释型** 语言：Python 不需要编译和链接，可以在一定程度上减少操作步骤。
- Python 是一门 **交互式** 语言：Python 解释器实现了交互式操作，可以直接在终端输入并执行指令。
- Python **易学易用**：Python 提供了大量的数据结构，也支持开发大型程序。
- Python **兼容性强**：Python 同时支持 Windows、macOS 和 Unix 操作系统。
- Python **实用性强**：从简单的输入输出到科学计算甚至于大型 WEB 应用，都可以写出适合的 Python 程序。
- Python **程序简洁、易读**：Python 代码通常比实现同种功能的其他语言的代码短。
- Python **支持拓展**：Python 会开发 C 语言程序（即 CPython），支持把 Python 解释器和用 C 语言开发的应用链接，用 Python 扩展和控制该应用。

#### 学习 Python 的注意事项

- 目前主要使用的 Python 版本是 Python 3.7 及以上的版本，Python 2 和 Python 3.6 及以前的 Python 3 已经**不被支持**，但仍被一些老旧系统与代码所使用。本文将介绍较新版本的 **Python**。如果遇到 Python 2 代码，可以尝试 [2to3](#) 程序将 Python 2 代码转换为 Python 3 代码。
- Python 的设计理念和语法结构与**一些其他语言的差异较大**，隐藏了许多底层细节，所以呈现出实用而优雅的风格。
- Python 是高度动态的解释型语言，因此其 **程序运行速度相对较慢**，尤其在使用其内置的 `for` 循环语句时。在使用 Python 时，应尽量使用 `filter`、`map` 等内置函数，或使用 [列表生成](#) 语法的手段来提高程序性能。

### 环境搭建

参见 [Python 3](#)。或者：

- Windows：也可以在 Microsoft Store 中免费而快捷地获取 Python。
- macOS/Linux：通常情况下，大部分的 Linux 发行版中已经自带了 Python。如果只打算学习 Python 语法，并无其它开发需求，不必另外安装 Python。

▼ 注意

在一些默认安装（指使用软件包管理器安装）Python 的系统（如 Unix 系统）中，应在终端中运行 `python3` 打开 Python 3 解释器。<sup>[1](#)</sup>

此外，也可以通过 `venv`、`conda`、`Nix` 等工具管理 Python 工具链和 Python 软件包，创建隔离的虚拟环境，避免出现依赖问题。

作为一种解释型语言，Python 的执行方式和 C++ 有所不同，这种差异在使用 IDE 编程时往往得不到体现，因此这里需要强调一下运行程序的不同方式。

当在命令行中键入 `python3` 或刚刚打开 IDLE 时，你实际进入了一种交互式的编程环境，也称「REPL」（「读取 - 求值 - 输出」循环），初学者可以在这里输入语句并立即看到结果，这让验证一些语法变得极为容易，我们也将后文中大量使用这种形式。

但若若要编写完整的程序，你最好还是新建一个文本文件（通常后缀为 `.py`），然后在命令行中执行 `python3 filename.py`，就能够运行代码看到结果了。

#### 一些平台提供的 Python 版本

| 系统名/版本           | python 版本               |
|------------------|-------------------------|
| Noi Linux 2.0    | 3.8.0, Include requests |
| Luogu 评测机        | 3.11.5, NumPy 1.25.2    |
| 基于 Hydro 的 OJ    | 3.8.0+ Include NumPy    |
| Ubuntu 22.04（内置） | 3.10.4                  |
| 微软商店             | 最新正式版                   |

▼ 注意

本表格在本文撰写时（2025/01/15）时有效，建议前往相关平台重新查证。

目前国内关于 **源码** 的镜像缓存主要是 [北京交通大学自由与开源软件镜像站](#) 和 [华为开源镜像站](#)，可以到那里尝试下载 Python 安装文件。

### 使用 pip 安装第三方库

Python 的生命力很大程度上来自于丰富的第三方库，编写一些实用程序时「调库」是常规操作，`pip` 是首选的安装第三方库的程序。自 Python 3.4 版本起，它被默认包含在 Python 二进制安装程序中。

`pip` 中的第三方库主要存储在 [Python 包索引 \(PyPI\)](#) 上，用户也可以指定其它第三方库的托管平台。使用方法可参照 [pypi 镜像使用帮助 - 清华大学开源软件镜像站](#) 等使用帮助。你可以在 [MirrorZ](#) 上获取更多 PyPI 镜像源。

▼ 使用清华大学开源镜像站安装一个包

```
1 pip install -i https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple &LTsome-package>
```

## 基本语法

Python 的语法简洁而易懂，也有许多官方和第三方文档与教程。这里仅介绍一些对 Oler 比较实用的语言特性，你可以在 [Python 文档](#) 和 [Python Wiki](#) 等网页上了解更多关于 Python 的教程。

### 注释

加入注释并不会对代码的运行产生影响，但加入注释可以使代码更加易懂易用。

```
1 # 用 # 字符开头的是单行注释
2
3 """
4 跨多行字符串会用三引号
5 （即三个单引号或三个双引号）
6 包裹，但也通常被用于注释
7 """
```

加入注释代码并不会对代码产生影响。我们鼓励加入注释来使代码更加易懂易用。

### 基本数据类型

#### 一切皆对象

在 Python 中，你无需事先声明变量名及其类型，直接赋值即可创建各种类型的变量：

```
1 >>> x = -3 # 语句结尾不用加分号
2 >>> x
3 -3
4 >>> f = 3.1415926535897932384626; f # 实在想加分号也可以，这里节省了一行
5 3.141592653589793
6 >>> s1 = "O"
7 >>> s1 # 在 Python 中双引号和单引号的作用相同
8 'O'
9 >>> b = 'A' == 65 # 'A' 和 65 不是一个数据类型，所以不相等
10 >>> b # True, False 首字母均大写
11 False
12 >>> True + 1 == 2 and not False != 0 # Python 中的表达式中大多使用单词，但是也支持符号
13 True
```

但这不代表 Python 没有类型的概念，实际上解释器会根据赋值或运算自动推断变量类型，你可以使用内置函数 `type()` 查看这些变量的类型：

```
1 >>> type(x)
2 &LTclass 'int'>
3 >>> type(f)
4 &LTclass 'float'>
5 >>> type(s1) # 请注意，不要给字符串起名为 str，否则 str 对象会被篡改
6 &LTclass 'str'>
7 >>> type(b)
8 &LTclass 'bool'>
```

#### ▼ 内置函数 是什么？

在 C/C++ 中，很多常用函数都分散在不同的头文件中，但 Python 的解释器内置了许多实用且通用的函数，你可以直接使用而无需注意它们的存在，但这也带来了小问题，这些内置函数的名称多为常见单词，你需要注意避免给自己的变量起相同的名字，否则可能会产生奇怪的结果。

正如我们所看到的，Python 内置有整数、浮点数、字符串和布尔类型，可以类比为 C++ 中的 `int`, `float`, `string` 和 `bool`。但有一些明显的不同之处，比如没有 `char` 字符类型，也没有 `double` 类型（但 `float` 其实对应 C 中的双精度），如果需要更精确的浮点运算，可以使用标准库中的 [decimal](#) 模块，如果需要用复数，Python 还内置了 `complex` 类型（而这也意味着最好不要给变量起名为 `complex`）。可以看到这些类型都以 `class` 开头，而这正是 Python 不同于 C++ 的关键之处，Python 程序中的所有数据都是由对象或对象间关系来表示的，函数是对象，类型本身也是对象：

```
1 >>> type(int)
2 &LTclass 'type'>
3 >>> type(pow) # 求幂次的内置函数，后文会介绍
4 &LTclass 'builtin_function_or_method'>
5 >>> type(type) # type() 也是内置函数，但有些特殊，感兴趣可自行查阅
6 &LTclass 'type'>
```

你或许会觉得这些概念一时难以理解且没有用处，所以我们暂时不再深入，在后文的示例中你或许能慢慢体会到，Python 的对象提供了强大的方法，我们在编程时应当优先考虑围绕对象而不是过程进行操作，这会让我们的代码显得更加紧凑明晰。

#### 数字运算

有人说，你可以把你系统里装的 Python 当作一个多用计算器，这是事实。

在交互模式下，你可以在提示符 `>>>` 后面输入一个表达式，就像其他大部分语言（如 C++）一样使用运算符 `+`、`-`、`*`、`/`、`%` 来对数字进行运算，也可以使用 `()` 来进行符合结合律的分组，读者可以自行试验，在这里我们仅展示与 C++ 差异较大的部分：

```

1 >>> 5.0 * 6 # 浮点数的运算结果是浮点数
2 30.0
3 >>> 15 / 3 # 与 C/C++ 不同，除法永远返回浮点 float 类型
4 5.0
5 >>> 5 / 100000 # 位数太多，结果显示成科学计数法形式
6 5e-05
7 >>> 5 // 3 # 使用整数除法（地板除）则会向下取整，输出整数类型
8 1
9 >>> -5 // 3 # 符合向下取整原则，注意这与 C/C++ 不同
10 -2
11 >>> 5 % 3 # 取模
12 2
13 >>> -5 % 3 # 负数取模结果一定是非负数，这点也与 C/C++ 不同，不过都满足 (a//b)*b+(a%b)==a
14 1
15 >>> x = abs(-1e4) # 求绝对值的内置函数
16 >>> x += 1 # 没有自增/自减运算符
17 >>> x # 科学计数法默认为 float
18 10001.0

```

在上面的实践中可以发现，除法运算 (/) 永远返回浮点类型（在 Python 2 中返回整数）。如果你想要整数或向下取整的结果的话，可以使用整数除法 (//)。同样的，你也可以像 C++ 中一样，使用模 (%) 来计算余数，科学计数法的形式也相同。

特别地，Python 用 \*\* 即可进行幂运算，还通过内置的 pow(a, b, mod) 提供了 [快速幂](#) 的高效实现。

```

1 >>> 3 ** 4 # 幂运算
2 81
3 >>> 2 ** 512
4 13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427690031858186486050853753882811946569946433649006084096
5 >>> pow(2, 512, int(1e4)) # 即 2**512 % 10000 的快速实现，1e4 是 float 所以要转 int
6 4096
7 >>> 2048 ** 2048 # 在IDLE里试试大整数?
8 >>> 0.1 + 0.1 + 0.1 - 0.3 == 0. # 和 C/C++ 一样需要注意浮点数不能直接判相等
9 False

```

## 数据类型判断

对于一个变量，可以使用 type(object) 返回变量的类型，例如 type(8) 和 type('a') 的值分别为 &LTclass 'int'> 和 &LTclass 'str'>。

## 基本输入输出

Python 中的输入输出主要通过内置函数 input() 和 print() 完成，print() 的用法十分符合直觉：

```

1 >>> a = [1,2,3]; print(a[-1]) # 打印时默认末尾换行
2 3
3 >>> print(ans[0], ans[1]) # 可以输出任意多个变量，默认以空格间隔
4 1 2
5 >>> print(a[0], a[1], end='') # 令 end='', 使末尾不换行
6 1 2>>>
7 >>> print(a[0], a[1], sep=', ') # 令 sep=', ', 改变间隔样式
8 1, 2
9 >>> print(str(a[0]) + ', ' + str(a[1])) # 输出同上，但是手动拼接成一整个字符串

```

input() 函数的行为接近 C++ 中的 getline()，即将一整行作为字符串读入，且末尾没有换行符。

```

1 >>> s = input('请输入一串数字: '); s # 自己调试时可以向 input() 传入字符串作为提示
2 请输入一串数字: 1 2 3 4 5 6
3 '1 2 3 4 5 6'

```

## 字符串

Python 3 提供了强大的基于 [Unicode](#) 的字符串类型，使用起来和 C++ 中的 string 类似，一些概念如转义字符也都相通，除了加号拼接和索引访问，还额外支持数乘 \* 重复字符串，和 in 操作符。

```

1 >>> s1 = "O" # 单引号和双引号都能包起字符串，有时可节省转义字符
2 >>> s1 += 'I-Wiki' # 为和 C++ 同步建议使用双引号
3 >>> 'OI' in s1 # 检测子串很方便
4 True
5 >>> len(s1) # 类似 C++ 的 s.length(), 但更通用
6 7
7 >>> s2 = "" 感谢你的阅读
8 ... 欢迎参与贡献!
9 "" # 使用三重引号的字符串可以跨越多行
10 >>> s1 + s2
11 'OI-Wiki 感谢你的阅读\n欢迎参与贡献!'
12 >>> print(s1 + s2) # 这里使用了 print() 函数打印字符串
13 OI-Wiki 感谢你的阅读
14 欢迎参与贡献!
15 >>> s2[2] * 2 + s2[3] + s2[-1] # 负数索引从右开始计数，加上 len(s)，相当于模 n 的剩余类环
16 '谢谢你!'
17 >>> s1[0] = 'o' # str 是不可变类型，不能原地修改，其实 += 也是创建了新的对象
18 Traceback (most recent call last):
19   File "&LTstdin>", line 1, in &LTmodule>
20 TypeError: 'str' object does not support item assignment

```

Python 支持多种复合数据类型，可将不同值组合在一起。最常用的 `list`，类型是用方括号标注、逗号分隔的一组值。例如，`[1, 2, 3]` 和 `['a', 'b', 'c']` 都是列表。

除了索引，字符串还支持 *切片*，它的设计非常精妙，格式为 `s[左闭索引:右开索引:步长]`：

```

1 >>> s = 'OI-Wiki 感谢你的阅读\n欢迎参与贡献!'
2 >>> s[:8] # 省略左闭索引则从头开始
3 'OI-Wiki '
4 >>> s[8:14] # 左闭右开设计的妙处，长度为 14-8=6，还和上一个字符串无缝衔接
5 '感谢你的阅读'
6 >>> s[-4:] # 省略右开索引则直到结尾
7 '与贡献!'
8 >>> s[8:14:2] # 步长为2
9 '感你阅'
10 >>> s[::-1] # 步长为 -1 时，获得了反转的字符串
11 '!献贡与参迎欢\n阅读的你感谢 ikiW-IO'
12 >>> s # 但原来的字符串并未改变
13 'OI-Wiki 感谢你的阅读\n欢迎参与贡献!'

```

在最新的 Python 3 版本中，字符串是以 Unicode 编码的，也就是说，Python 的字符串支持多语言。<sup>2</sup>在 Python 中，可以对一个 Unicode 字符使用内置函数 `ord()` 将其转换为对应的 Unicode 编码，逆向的转换使用内置函数 `chr()`。C/C++ 中 `char` 类型也可以和对应的 ASCII 码互转。

如果想把数字转换成对应的字符串，可以使用内置函数 `str()`，反之可以使用 `int()` 和 `float()`，你可以类比为 C/C++ 中的强制类型转换，但括号不是加在类型上而是作为函数的一部分括住参数。

Python 的字符串类型提供了许多强大的方法，包括计算某字符的索引与出现次数，转换大小写等等，这里就不一一列举，强烈建议查看 [官方文档](#) 熟悉常用方法，遇到字符串操作应当首先考虑使用这些方法而非自力更生。

## 创建数组

从 C++ 转过来的同学可能很迷惑怎么在 Python 中创建数组，这里就介绍在 Python 开「数组」的语法，需要强调我们介绍的其实是几种 [序列类型](#)，和 C 的数组有着本质区别，而更接近 C++ 中的 `vector`。

### 使用 list

列表 (`list`) 大概是 Python 中最常用也最强大的序列类型，列表中可以存放任意类型的元素，包括嵌套的列表，这符合数据结构中「广义表」的定义。请注意不要将其与 C++ STL 中的双向链表 [list](#) 混淆，故本文将使用「列表」而非 `list` 以免造成误解。

```

1 >>> [] # 创建空列表，注意列表使用方括号
2 []
3 >>> nums = [0, 1, 2, 3, 5, 8, 13]; nums # 初始化列表，注意整个列表可以直接打印
4 [0, 1, 2, 3, 5, 8, 13]
5 >>> nums[0] = 1; nums # 支持索引访问，支持修改元素
6 [1, 1, 2, 3, 5, 8, 13]
7 >>> nums.append(nums[-2]+nums[-1]); nums # append() 同 vector 的 push_back(), 也都没有返回值
8 [1, 1, 2, 3, 5, 8, 13, 21]
9 >>> nums.pop() # 弹出并返回末尾元素，可以当栈使用；其实还可指定位置，默认是末尾
10 21
11 >>> nums.insert(0, 1); nums # 同 vector 的 insert(position, val)
12 [1, 1, 1, 2, 3, 5, 8, 13]
13 >>> nums.remove(1); nums # 按值移除元素（只删第一个出现的），若不存在则抛出错误
14 [1, 1, 2, 3, 5, 8, 13]
15 >>> len(nums) # 求列表长度，类似 vector 的 size(), 但 len() 是内置函数
16 7
17 >>> nums.reverse(); nums # 原地逆置
18 [13, 8, 5, 3, 2, 1, 1]
19 >>> sorted(nums) # 获得排序后的列表
20 [1, 1, 2, 3, 5, 8, 13]
21 >>> nums # 但原来的列表并未排序
22 [13, 8, 5, 3, 2, 1, 1]
23 >>> nums.sort(); nums # 原地排序，可以指定参数 key 作为排序标准
24 [1, 1, 2, 3, 5, 8, 13]
25 >>> nums.count(1) # 类似 std::count()
26 2
27 >>> nums.index(1) # 返回值首次出现项的索引号，若不存在则抛出错误
28 0
29 >>> nums.clear(); nums # 同 vector 的 clear()

```

以上示例展现了列表与 `vector` 的相似之处，`vector` 中常用的操作一般也都能在列表中找到对应方法，不过某些方法如 `len()`, `sorted()` 会以内置函数的面目出现，而 STL 算法中的函数如 `find()`, `count()`, `max_element()`, `sort()`, `reverse()` 在 Python 中又成了对象的方法，使用时需要注意区分，更多方法请参见官方文档的 [列表详解](#)。下面将展示列表作为 Python 的基本序列类型的一些强大功能：

Python 支持多种复合数据类型，可将不同值组合在一起。最常用的 `list`，类型是用方括号标注、逗号分隔的一组值。例如，`[1, 2, 3]` 和 `['a', 'b', 'c']` 都是列表。

```

1 >>> lst = [1, '1'] + ["2", 3.0] # 列表直接相加生成一个新列表
2 >>> lst # 这里存放不同的类型只是想说明可以这么做，但这不是好的做法
3 [1, '1', '2', 3.0]
4 >>> 3 in lst # 实用的成员检测操作，字符串也有该操作且还支持子串检测
5 True
6 >>> [1, '1'] in lst # 仅支持单个成员检测，不会发现「子序列」
7 False
8 >>> lst[1:3] = [2, 3]; lst # 切片并赋值，原列表被修改
9 [1, 2, 3, 3.0]
10 >>> lst[::-1] # 获得反转后的新列表
11 [3.0, 3, 2, 1]
12 >>> lst *= 2; lst # 数乘拼接
13 [1, 2, 3, 3.0, 1, 2, 3, 3.0]
14 >>> del lst[4:]; lst # 也可写 lst[4:] = [], del 语句不止可以用于删除序列中元素
15 [1, 2, 3, 3.0]

```

以上示例展现了列表作为序列的一些常用操作，可以看出许多操作如切片是与字符串相通的，但字符串是「不可变序列」而列表是「可变序列」，故可以通过切片灵活地修改列表。在 C/C++ 中我们往往会通过循环处理字符数组，下面将展示如何使用 [「列表推导式」](#) 在字符串和列表之间转换：

```

1 >>> # 建立一个 [65, 70) 区间上的整数数组，range 也是一种类型，可看作左闭右开区间，第三个参数为步长可省略
2 >>> nums = list(range(65,70)) # 记得 range 外面还要套一层 list()
3 [65, 66, 67, 68, 69]
4 >>> lst = [chr(x) for x in nums] # 列表推导式的典型结构，[exp for var in iterable if cond]
5 >>> lst # 上两句可以合并成 [chr(x) for x in range(65,70)]
6 ['A', 'B', 'C', 'D', 'E']
7 >>> s = ''.join(lst); s # 用空字符串 '' 拼接列表中的元素生成新字符串
8 'ABCDE'
9 >> list(s) # 字符串生成字符列表
10 ['A', 'B', 'C', 'D', 'E']
11 >>> # 如果你不知道有 s.lower() 方法就可能写出下面这样新瓶装旧酒的表达式
12 >>> ''.join([chr(ord(ch) - 65 + 97) for ch in s if ch >= 'A' and ch <= 'Z'])
13 'abcde'

```

下面演示一些在 OI 中更常见的场景，比如二维「数组」：

```

1 >>> vis = [[0] * 3] * 3 # 开一个 3*3 的全 0 数组
2 >>> vis
3 [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
4 >>> vis[0][0] = 1; vis # 怎么会把其他行也修改了?
5 [[1, 0, 0], [1, 0, 0], [1, 0, 0]]
6 >>> # 先来看一下一维列表的赋值
7 >>> a1 = [0, 0, 0]; a2 = a1; a3 = a1[:] # 列表也可以直接被赋给新的变量
8 >>> a1[0] = 1; a1 # 修改列表 a1, 似乎正常
9 [1, 0, 0]
10 >>> a2 # 怎么 a2 也被改变了
11 [1, 0, 0]
12 >>> a3 # a3 没有变化
13 [0, 0, 0]
14 >>> id(a1) == id(a2) and id(a1) != id(a3) # 内置函数 id() 给出对象的「标识值」, 可类比为地址, 地址相同说明是一个对象
15 True
16 >>> vis2 = vis[:] # 拷贝一份二维列表
17 >>> vis[0][1] = 2; vis # vis 会被批量修改
18 >>> [[1, 2, 0], [1, 2, 0], [1, 2, 0]]
19 >>> vis2 # 但 vis2 是切片拷贝还是被改了
20 >>> [[1, 2, 0], [1, 2, 0], [1, 2, 0]]
21 >>> id(vis) != id(vis2) # vis 和 vis2 不是一个对象
22 True
23 >>> # vis2 虽然不是 vis 的引用, 但其中对应行都指向相同的对象
24 >>> [id(vis[i]) == id(vis2[i]) for i in range(3)]
25 [True, True, True]
26 >>> # 回看二维列表自身
27 >>> [id(x) for x in vis] # 具体数字和这里不一样但三个值一定相同, 说明是三个相同对象
28 [139760373248192, 139760373248192, 139760373248192]

```

其实有一个重要的事实, Python 中赋值只传递了引用而非创建新值, 你可以创建不同类型的变量并赋给新变量, 验证发现二者的标识值是相同的, 只不过直到现在我们才介绍了列表这种可变类型, 而给数字、字符串这样的不可变类型赋新值时实际上创建了新的对象, 故而前后两个变量互不干扰。但列表是可变类型, 所以我们修改一个列表的元素时, 另一个列表由于指向同一个对象所以也被修改了。创建二维数组也是类似的情况, 示例中用乘法创建二维列表相当于把 `[0]*3` 这个一维列表重复了 3 遍, 所以涉及其中一个列表的操作会同时影响其他两个列表。更不幸的是, 在将二维列表赋给其他变量的时候, 就算用切片来拷贝, 也只是「浅拷贝」, 其中的元素仍然指向相同的对象, 解决这个问题需要使用标准库中的 [deepcopy](#), 或者尽量避免整个赋值二维列表。不过还好, 创建二维列表时避免创建重复的列表还是比较简单, 只需使用「列表推导式」:

```

1 >>> vis1 = [[0] * 3 for _ in range(3)] # 把用不到的循环计数变量设为下划线 _ 是一种惯例
2 >>> # 但在 REPL 中 _ 默认指代上一个表达式输出的结果, 故也可使用双下划线
3 >>> vis1
4 [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
5 >>> [id(x) for x in vis1] # 具体数字和这里不一样但三个值一定不同, 说明是三个不同对象
6 [139685508981248, 139685508981568, 139685508981184]
7 >>> vis1[0][0] = 1
8 [[1, 0, 0], [0, 0, 0], [0, 0, 0]]
9 >>> a2[0][0] = 10 # 访问和赋值二维数组

```

我们未讲循环的用法就先介绍了列表推导式, 这是由于 Python 是高度动态的解释型语言, 因此其程序运行有大量的额外开销。尤其是 **for** 循环在 **Python** 中运行的奇慢无比。因此在使用 Python 时若想获得高性能, 尽量使用使用列表推导式, 或者 `filter,map` 等内置函数直接操作整个序列来避免循环, 当然这还是要根据具体问题而定。

## 使用 NumPy

### ► 什么是 NumPy

下面的代码将介绍如何利用 NumPy 建立多维数组并进行访问。

```

1 >>> import numpy as np # 请自行搜索 import 的意义和用法
2 >>> np.empty(3) # 容量为 3 的空数组, 注意没有初始化为 0
3 array([0.00000000e+000, 0.00000000e+000, 2.01191014e+180])
4 >>> np.zeros((3, 3)) # 开 3*3 的数组, 并初始化为 0
5 array([[0., 0., 0.],
6        [0., 0., 0.],
7        [0., 0., 0.]])
8 >>> a1 = np.zeros((3, 3), dtype=int) # 开 3*3 的整数数组
9 >>> a1[0][0] = 1 # 访问和赋值
10 >>> a1[0, 0] = 1 # 更友好的语法
11 >>> a1.shape # 数组的形状
12 (3, 3)
13
14 >>> a1[:2, :2] # 取前两行、前两列构成的子阵, 无拷贝
15 array([[1, 0],
16        [0, 0]])
17
18 >>> a1[:, [0, 2]] # 获取第 1、3 列, 无拷贝
19 array([[1, 0],
20        [0, 0],
21        [0, 0]])
22 >>> np.max(a1) # 获取数组最大值
23 1
24 >>> a1.flatten() # 将数组展平
25 array([1, 0, 0, 0, 0, 0, 0, 0, 0])
26
27 >>> np.sort(a1, axis = 1) # 沿行方向对数组进行排序, 返回排序结果
28 array([[0, 0, 1],
29        [0, 0, 0],
30        [0, 0, 0]])
31 >>> a1.sort(axis = 1) # 沿行方向对数组进行原地排序

```

## 使用 array

[array](#) 是 Python 标准库提供了一种高效数值数组, 可以紧凑地表示基本类型值的数组, 但不支持数组嵌套, 也很少见到有人使用它, 这里只是顺便提一下。

若无特殊说明, 后文出现「数组」一般指「列表」。

## 输入输出

Python 中的输入输出主要通过内置函数 `input()` 和 `print()` 完成。前文已经介绍过, 下面介绍进阶用法。

### 格式化输出

算法竞赛中通常只涉及到基本的数值和字符串输出, `print()` 已基本足够, 只有当涉及到浮点位数时需要用到格式化字符串输出。格式化有三种方法, 第一种也是最老旧的方法是使用 `printf()` 风格的 `%` 操作符; 另一种是利用 [format 函数](#); 第三种是 Python 3.6 新增的 [f-string](#), 最为简洁, 但不保证考场中的 Python 版本足够新。详细丰富的说明可以参考 [这个网页](#), 尽管更推荐使用 `format()` 方法, 但为了获得与 C 接近的体验, 下面仅演示与 `printf()` 类似的老式方法:

```

1 >>> pi = 3.1415926; print('%4f' % pi) # 格式为 %[flags][width][.precision]type
2 3.1416
3 >>> '%4f - %8f = %d' % (pi, 0.1416, 3) # 右边多个参数用 () 括住, 后面会看到其实是「元组」
4 '3.1416 - 0.141600 = 3'

```

### split() 函数

`input()` 函数的行为接近 C++ 中的 `getline()`, 即将一整行作为字符串读入, 且末尾没有换行符, 但在算法竞赛中, 常见的输入形式是一行输入多个数值, 因此就需要使用字符串的 `split()` 方法并搭配列表推导式得到存放数值类型的列表, 下面以输入  $n$  个数求平均值为例演示输入  $n$  个数得到「数组」的方法:

```

1 >>> s = input('请输入一串数字: '); s # 自己调试时可以向 input() 传入字符串作为提示
2 请输入一串数字: 1 2 3 4 5 6
3 '1 2 3 4 5 6'
4 >>> a = s.split(); a
5 ['1', '2', '3', '4', '5', '6']
6 >>> a = [int(x) for x in a]; a
7 [1, 2, 3, 4, 5, 6]
8 >>> # 以上输入过程可写成一行 a = [int(x) for x in input().split()]
9 >>> sum(a) / len(a) # sum() 是内置函数
10 3.5

```

有时题目会在每行输入固定几个数, 比如边的起点、终点、权重, 如果只用上面提到的方法就只能每次读入数组然后根据下标赋值, 这时可以使用 Python 的「拆包」特性一次赋值多个变量:

```

1 >>> u, v, w = [int(x) for x in input().split()]
2 1 2 4
3 >>> print(u,v,w)
4 1 2 4

```

题目中经常遇到输入  $N$  行的情况, 可我们还没有讲最基本的循环语句, 但 Python 强大的序列操作能在不使用循环的情况下应对多行输入, 下面假设将各条边的起点、终点、权值分别读入三个数组:

```

1 >>> N = 4; mat = [[int(x) for x in input().split()] for i in range(N)]
2 1 3 3
3 1 4 1
4 2 3 4
5 3 4 1
6 >>> mat # 先按行读入二维数组
7 [[1, 3, 3], [1, 4, 1], [2, 3, 4], [3, 4, 1]]
8 >>> u, v, w = map(list, zip(*mat))
9 # *将 mat 解包得到里层的多个列表
10 # zip() 将多个列表中对应元素聚合成元组, 得到一个迭代器
11 # map(list, iterable) 将序列中的元素 (这里为元组) 转成列表
12 >>> print(u, v, w) # 直接将 map() 得到的迭代器拆包, 分别赋值给 u, v, w
13 [1, 1, 2, 3] [3, 4, 3, 4] [3, 1, 4, 1]

```

上述程序实际上相当于先读入一个  $N$  行  $3$  列的矩阵, 然后将其转置成  $3$  行  $N$  列的矩阵, 也就是外层列表中嵌套了  $3$  个列表, 最后将代表这起点、终点、权值的  $3$  个列表分别赋值给  $u, v, w$ 。内置函数 [zip\(\)](#) 可以将多个等长序列中的对应元素拼接在「元组」内, 得到新序列。而 `map()` 其实是函数式编程的一种操作, 它将一个给定函数作用于 `zip()` 所产生序列的元素, 这里就是用 `list()` 将元组变成列表。你可以自行练习使用 `*` 和 [zip\(\)](#), [map\(\)](#) 以理解其含义。需要注意的是 Python 3 中 `zip()` 和 `map()` 创建的不再返回列表而是返回迭代器, 这里暂不解释它们之间的异同, 你可以认为迭代器可以产生列表中的各个元素, 用 `list()` 套住迭代器就能生成列表。

## 控制流程

尽管我们已经学习了 Python 的许多特性, 但到目前为止我们展示的 Python 代码都是单行语句, 这掩盖了 Python 和 C 在代码风格上的重大差异: 首先, Python 中不用 `{}` 而是用缩进表示块结构, 如果缩进没有对齐会直接报错, 如果 `tab` 和 空格混用也会报错; 其次, 块结构开始的地方比如 `if` 和 `for` 语句的行末要有冒号 `:`。这有助于代码的可读性, 但你也可能怀念 C 那种自由的体验, 毕竟如果复制粘贴时因为丢失缩进而不得不手动对齐是很恼人的。

## 循环结构

列表推导式能在一行内高效地完成批量操作, 但有时为了压行我们已经显得过分刻意, 许多场景下还是只能使用循环结构, 所以我们再以读入多行数据为例展示 Python 中的循环是如何编写的:

```

1 # 请注意从现在开始我们不再使用 REPL, 请自行复制多行数据
2 u, v, w = ([ ] for i in range(3)) # 多变量赋值
3 for i in range(4): # 这里假设输入 4 行数据
4     _u, _v, _w = [int(x) for x in input().split()]
5     u.append(_u), v.append(_v), w.append(_w)
6     # 不可进行类似 cin >> u[i] >> v[i] >> w[i] 的操作, 因为必定超出列表当前的长度
7     # 当然你可以选择初始化为 MAXN 的全 0 列表, 不过需要记住真实长度并删掉多余元素
8 print(u, v, w)

```

需要注意, Python 中的 `for` 循环和 C/C++ 有较大的差别, 其作用类似 C++ 11 引入的「[基于范围的循环](#)」, 实质是迭代序列中的元素, 比如编写循环遍历数组下标需要迭代 `range(len(lst))`, 而非真正定义起始和终止条件, 所以使用起来并没有 C/C++ 灵活。

下面再用 `while` 循环展示行数不定的情况下如何输入:

```

1 u, v, w = [ ], [ ], [ ] # 多变量赋值, 其实同上
2 s = input() # 注意 Python 中赋值语句不能放在条件表达式中
3 while s: # 不能像 C 那样 while(!scanf())
4     # 用切片拼接避免了 append(), 注意列表推导式中又嵌套了列表
5     u[len(u) :], v[len(v) :], w[len(w) :] = [[int(x)] for x in s.split()]
6     s = input()
7 # Python 3.8 引入了 walrus operator 海象运算符后, 你可以节省两行, 但考场环境很可能不支持
8 while s := input():
9     u[len(u) :], v[len(v) :], w[len(w) :] = [[int(x)] for x in s.split()]
10 print(u, v, w)

```

## 选择结构

和 C/C++ 大同小异, 一些形式上的差别都在下面的示例中有所展示, 此外还需注意条件表达式中不允许使用赋值运算符 (Python 3.8 以上可用 `:=`), 以及 [没有 switch 语句](#)。

```

1 # 条件表达式两侧无括号
2 if 4 >= 3 > 2 and 3 != 5 == 5 != 7:
3     print("关系运算符可以连续使用")
4     x = None or [ ] or -2
5     print("%&& || !", "与 或 非", "and or not", sep="\n")
6     print("善用 and/or 可节省行数")
7     if not x:
8         print("负数也是 True, 不执行本句")
9     elif x & 1:
10        print("用 elif 而不是 else if\n" "位运算符与 C 相近, 偶数&1 得 0, 不执行本句")
11    else:
12        print("也有三目运算符") if x else print("注意结构")

```

## 异常处理

尽管 C++ 中有 [try 块](#) 用于异常处理, 但竞赛中一般从不使用, 而 Python 中常见的是 [EAFP](#) 风格, 故而代码中可能大量使用 [try-except](#) 语句, 在后文介绍 `dict` 这一结构时还会用到, 这里展示:



```

1 s = "OI-wiki"
2 pat = "NOIP"
3 x = s.find(pat) # find() 找不到返回 -1
4 try:
5     y = s.index(pat) # index() 找不到则抛出错误
6     print(y) # 这句被跳过
7 except ValueError:
8     print("没找到")
9     try:
10        print(y) # 此时 y 并没有定义, 故又会抛出错误
11    except NameError as e:
12        print("无法输出 y")
13        print("原因:", e)

```

## 文件读写

Python 内置函数 [open\(\)](#) 用于文件读写, 为了防止读写过程中出错导致文件未被正常关闭, 这里只介绍使用 [with](#) 语句的安全读写方法:

```

1 a = []
2 with open("in.txt") as f:
3     N = int(f.readline()) # 读入第一行的 N
4     a[len(a) :] = [[int(x) for x in f.readline().split()] for i in range(N)]
5
6 with open("out.txt", "w") as f:
7     f.write("\n")

```

关于文件读写的函数有很多, 分别适用于不同的场景, 由于 OI 赛事尚不支持使用 Python, 这里从略。

## 内置容器

Python 内置了许多强大的容器类型, 只有熟练使用并了解其特点才能真正让 Python 在算法竞赛中有用武之地, 除了上面详细介绍的 [list](#) (列表), 还有 [tuple](#) (元组)、[dict](#) (字典) 和 [set](#) (集合) 这几种类型。

元组可以简单理解成不可变的列表, 不过还需注意「不可变」的内涵, 如果元组中的某元素是可变类型比如列表, 那么仍可以修改该列表的值, 元组中存放的是对列表的引用所以元组本身并没有改变。元组的优点是开销较小且「[可哈希](#)」, 后者在创建字典和集合时非常有用。

```

1 tup = tuple([[1, 2], 4]) # 由列表得到元组
2 # 等同于 tup = ([1,2], 4)
3 tup[0].append(3)
4 print(tup)
5 a, b = 0, "I-Wiki" # 多变量赋值其实是元组拆包
6 print(id(a), id(b))
7 b, a = a, b
8 print(id(a), id(b)) # 你应该会看到 a, b 的 id 值现在互换了
9 # 这更说明 Python 中, 变量更像是名字, 赋值只是让其指代对象

```

字典就像 C++ STL 中的 [map](#) (请注意和 Python 中内置函数 [map\(\)](#) 区分) 用于存储键值对, 形式类似 [JSON](#), 但 JSON 中键必须是字符串且以双引号括住, 字典则更加灵活强大, 可哈希的对象都可作为字典的键。需要注意 Python 几次版本更新后字典的特性有了较多变化, 包括其中元素的顺序等, 请自行探索。

```

1 dic = {"key": "value"} # 基本形式
2 dic = {chr(i): i for i in range(65, 91)} # 大写字母到对应 ASCII 码的映射, 注意断句
3 dic = dict(zip([chr(i) for i in range(65, 91)], range(65, 91))) # 效果同上
4 dic = {dic[k]: k for k in dic} # 将键值对逆转, for k in dic 迭代其键
5 dic = {v: k for k, v in dic.items()} # 和上行作用相同, dic.items() 以元组存放单个键值对
6 dic = {
7     k: v for k, v in sorted(dic.items(), key=lambda x: -x[1])
8 } # 字典按值逆排序, 用到了 lambda 表达式
9
10 print(dic["A"]) # 返回 dic 中以 'A' 为键的项, 这里值为 65
11 dic["a"] = 97 # 将 d[key] 设为 value, 字典中原无 key 就是直接插入
12 if "b" in dic: # LBYL(Look Before You Leap) 风格
13     print(dic["b"]) # 若字典中无该键则会出错, 故先检查
14 else:
15     dic["b"] = 98
16
17 # 经典场景 统计出现次数
18 # 新键不存在于原字典, 需要额外处理
19 try: # EAFP (Easier to Ask for Forgiveness than Permission) 风格
20     cnter[key] += 1
21 except KeyError:
22     cnter[key] = 1

```

集合就像 C++ STL 中的 [set](#), 不会保存重复的元素, 可以看成只保存键的字典。需要注意集合和字典都用 `{}` 括住, 不过单用 `{}` 会创建空字典而不是空集合, 这里就不再给出示例。

## 编写函数

Python 中定义函数无需指定参数类型和返回值类型, 无形中为 OI 选手减少了代码量

```

1 def add(a, b):
2     return a + b # 动态类型的优势, a 和 b 也可以是字符串
3
4
5 def add_no_swap(a, b):
6     print("in func #1:", id(a), id(b))
7     a += b
8     b, a = a, b
9     print("in func #2:", id(a), id(b)) # a, b 已交换
10    return a, b # 返回多个值, 其实就是返回元组, 可以拆包接收
11
12
13 lst1 = [1, 2]
14 lst2 = [3, 4]
15 print("outside func #1:", id(lst1), id(lst2))
16 add_no_swap(lst1, lst2)
17 # 函数外 lst1, lst2 并未交换
18 print("outside func #2:", id(lst1), id(lst2))
19 # 不过值确实已经改变
20 print(lst1, lst2)

```

## 默认参数

Python 中函数的参数非常灵活, 有关键字参数、可变参数等, 但在算法竞赛中这些特性的用处并不是很大, 这里只介绍一下默认参数, 因为 C++ 中也有默认参数, 且在 Python 中使用默认参数很有可能遇到坑。

```

1 def append_to(element, to=[]):
2     to.append(element)
3     return to
4
5
6 lst1 = append_to(12)
7 lst2 = append_to(42)
8 print(lst1, lst2)
9
10 # 你可能以为输出是 [12] [42]
11 # 但运行结果其实是 [12] [12, 42]
12
13
14 # 这是因为默认参数的值仅仅在函数定义的时候赋值一次
15 # 默认参数的值应该是不可变对象, 使用 None 占位是一种最佳实践
16 def append_to(element, to=None):
17     if to is None:
18         to = []
19     to.append(element)
20     return to

```

## 类型标注

Python 是一个动态类型检查的语言, 以灵活但隐式的方式处理类型, Python 解释器仅仅在运行时检查类型是否正确, 并且允许在运行时改变变量类型, 俗话说「动态类型一时爽, 代码重构火葬场」, 程序中的一些错误可能在运行时才会暴露:

```

1 >>> if False:
2 ...     1 + "two" # This line never runs, so no TypeError is raised
3 ... else:
4 ...     1 + 2
5 ...
6 3
7
8 >>> 1 + "two" # Now this is type checked, and a TypeError is raised
9 TypeError: unsupported operand type(s) for +: 'int' and 'str'

```

Python 3.5 后引入了类型标注, 允许设置函数参数和返回值的类型, 但只是作为提示, 并没有实际的限制作用, 需要静态检查工具才能排除这类错误 (例如 [PyCharm](#) 和 [Mypy](#)), 所以显得有些鸡肋, 对于 Oler 来说更是只需了解, 可按如下方式对函数的参数和返回值设置类型标注:

```

1 def headline(
2     text, # type: str
3     width=80, # type: int
4     fill_char=" ", # type: str
5 ): # type: (...) -> str
6     return f"{text.title()}.center(width, fill_char)"
7
8
9 print(headline("type comments work", width=40))

```

除了函数参数, 变量也是可以类型标注的, 你可以通过调用 `__annotations__` 来查看函数中所有的类型标注。变量类型标注赋予了 Python 静态语言的性质, 即声明与赋值分离:

```
1 >>> nothing: str
2 >>> nothing
3 NameError: name 'nothing' is not defined
4
5 >>> __annotations__
6 {'nothing': <class 'str'>}
```

## 装饰器

装饰器是一个函数，接受一个函数或方法作为其唯一的参数，并返回一个新函数或方法，其中整合了修饰后的函数或方法，并附带了一些额外的功能。简而言之，可以在不修改函数代码的情况下，增加函数的功能。相关知识可以参考 [官方文档](#)。

部分装饰器在竞赛中非常实用，比如 [lru\\_cache](#)，可以为函数自动增加记忆化的能力，在递归算法中非常实用：

```
@lru_cache(maxsize=128,typed=False)
```

- 传入的参数有 2 个：maxsize 和 typed，如果不传则 maxsize 的默认值为 128，typed 的默认值为 False。
- 其中 maxsize 参数表示的是 LRU 缓存的容量，即被装饰的方法的最大可缓存结果的数量。如果该参数值为 128，则表示被装饰方法最多可缓存 128 个返回结果；如果 maxsize 传入为 None 则表示可以缓存无限个结果。
- 如果 typed 设置为 True，不同类型的函数参数将被分别缓存，例如，f(3) 和 f(3.0) 会缓存两次。

以下是使用 lru\_cache 优化计算斐波那契数列的例子：

```
1 @lru_cache(maxsize=None)
2 def fib(n):
3     if n < 2:
4         return n
5     return fib(n - 1) + fib(n - 2)
```

## 常用内置库

在这里介绍一些写算法可能用到的内置库，具体用法可以自行搜索或者阅读 [官方文档](#)。

| 库名                          | 用途             |
|-----------------------------|----------------|
| <a href="#">array</a>       | 定长数组           |
| <a href="#">argparse</a>    | 命令行参数处理        |
| <a href="#">bisect</a>      | 二分查找           |
| <a href="#">collections</a> | 有序字典、双端队列等数据结构 |
| <a href="#">fractions</a>   | 有理数            |
| <a href="#">heapq</a>       | 基于堆的优先级队列      |
| <a href="#">io</a>          | 文件流、内存流        |
| <a href="#">itertools</a>   | 迭代器            |
| <a href="#">math</a>        | 数学函数           |
| <a href="#">os.path</a>     | 系统路径等          |
| <a href="#">random</a>      | 随机数            |
| <a href="#">re</a>          | 正则表达式          |
| <a href="#">struct</a>      | 转换结构体和二进制数据    |
| <a href="#">sys</a>         | 系统信息           |

## 从例题对比 C++ 与 Python

► [例题 洛谷 P4779【模板】单源最短路径（标准版）](#)

### 声明常量



C++Python

```
1 #include <iostream>
2 #include <string>
3 #include <queue>
4 #include <vector>
5 using namespace std;
6 constexpr int N = 1e5 + 5, M = 2e5 + 5;

1 try: # 引入优先级队列模块
2     import Queue as pq # python version < 3.0
3 except ImportError:
4     import queue as pq # python3.*
5
6 N = int(1e5 + 5)
7 M = int(2e5 + 5)
8 INF = 0x3F3F3F3F
```

### 声明前向星结构体和其它变量



## C++Python

```
1 struct qxx {
2     int nex, t, v;
3 };
4
5 qxx e[M];
6 int h[N], cnt;
7
8 void add_path(int f, int t, int v) { e[++cnt] = qxx{h[f], t, v}, h[f] = cnt; }
9
10 using pii = pair<int, int>;
11 priority_queue<pii, vector<pii>, greater<pii>> q;
12 int dist[N];

1 class qxx: # 前向星类 (结构体)
2     def __init__(self):
3         self.nex = 0
4         self.t = 0
5         self.v = 0
6
7
8 e = [qxx() for i in range(M)] # 链表
9 h = [0 for i in range(N)]
10 cnt = 0
11
12 dist = [INF for i in range(N)]
13 q = pq.PriorityQueue() # 定义优先队列，默认第一元小根堆
14
15
16 def add_path(f, t, v): # 在前向星中加边
17     # 如果要修改全局变量，要使用 global 来声明
18     global cnt, e, h
19     # 调试时的输出语句，多个变量使用元组
20     # print("add_path(%d,%d,%d)" % (f,t,v))
21     cnt += 1
22     e[cnt].nex = h[f]
23     e[cnt].t = t
24     e[cnt].v = v
25     h[f] = cnt
```

## Dijkstra 算法



## C++Python

```
1 void dijkstra(int s) {
2     memset(dist, 0x3f, sizeof(dist));
3     dist[s] = 0, q.push(make_pair(0, s));
4     while (q.size()) {
5         pii u = q.top();
6         q.pop();
7         if (dist[u.second] < u.first) continue;
8         for (int i = h[u.second]; i; i = e[i].nex) {
9             const int &v = e[i].t, &w = e[i].v;
10            if (dist[v] <= dist[u.second] + w) continue;
11            dist[v] = dist[u.second] + w;
12            q.push(make_pair(dist[v], v));
13        }
14    }
15 }

1 def nextedgeid(u): # 生成器，可以用在 for 循环里
2     i = h[u]
3     while i:
4         yield i
5         i = e[i].nex
6
7
8 def dijkstra(s):
9     dist[s] = 0
10    q.put((0, s))
11    while not q.empty():
12        u = q.get() # get 函数会顺便删除堆中对应的元素
13        if dist[u[1]] < u[0]:
14            continue
15        for i in nextedgeid(u[1]):
16            v = e[i].t
17            w = e[i].v
18            if dist[v] <= dist[u[1]] + w:
19                continue
20            dist[v] = dist[u[1]] + w
21            q.put((dist[v], v))
```

## 主函数



## C++Python

```
1 int n, m, s;
2
3 int main() {
4     scanf("%d%d%d", &n, &m, &s);
5     for (int i = 1; i <= m; i++) {
6         int u, v, w;
7         scanf("%d%d%d", &u, &v, &w);
8         add_path(u, v, w);
9     }
10    dijkstra(s);
11    for (int i = 1; i <= n; i++) printf("%d ", dist[i]);
12    return 0;
13 }
```

  

```
1 if __name__ == "__main__":
2     # 一行读入多个整数。注意它会把整行都读进来
3     n, m, s = map(int, input().split())
4     for i in range(m):
5         u, v, w = map(int, input().split())
6         add_path(u, v, w)
7
8     dijkstra(s)
9
10    for i in range(1, n + 1):
11        print(dist[i], end=" ")
12
13    print()
```

## 完整代码



## C++Python

```
1 #include <LTestdio>
2 #include <LTeststring>
3 #include <LTqueue>
4 #include <LTvector>
5 using namespace std;
6 constexpr int N = 1e5 + 5, M = 2e5 + 5;
7
8 struct qxx {
9     int nex, t, v;
10 };
11
12 qxx e[M];
13 int h[N], cnt;
14
15 void add_path(int f, int t, int v) { e[++cnt] = qxx{h[f], t, v}, h[f] = cnt; }
16
17 using pii = pair<int, int>;
18 priority_queue<pii, vector<pii>, greater<pii>> q;
19 int dist[N];
20
21 void dijkstra(int s) {
22     memset(dist, 0x3f, sizeof(dist));
23     dist[s] = 0, q.push(make_pair(0, s));
24     while (q.size()) {
25         pii u = q.top();
26         q.pop();
27         if (dist[u.second] < u.first) continue;
28         for (int i = h[u.second]; i; i = e[i].nex) {
29             const int &v = e[i].t, &w = e[i].v;
30             if (dist[v] <= dist[u.second] + w) continue;
31             dist[v] = dist[u.second] + w;
32             q.push(make_pair(dist[v], v));
33         }
34     }
35 }
36
37 int n, m, s;
38
39 int main() {
40     scanf("%d%d%d", &n, &m, &s);
41     for (int i = 1; i <= m; i++) {
42         int u, v, w;
43         scanf("%d%d%d", &u, &v, &w);
44         add_path(u, v, w);
45     }
46     dijkstra(s);
47     for (int i = 1; i <= n; i++) printf("%d ", dist[i]);
48     return 0;
49 }
```

```

1 try: # 引入优先队列模块
2     import Queue as pq # python version < 3.0
3 except ImportError:
4     import queue as pq # python3.*
5
6 N = int(1e5 + 5)
7 M = int(2e5 + 5)
8 INF = 0x3F3F3F3F
9
10
11 class qxx: # 前向星类 (结构体)
12     def __init__(self):
13         self.nex = 0
14         self.t = 0
15         self.v = 0
16
17
18 e = [qxx() for i in range(M)] # 链表
19 h = [0 for i in range(N)]
20 cnt = 0
21
22 dist = [INF for i in range(N)]
23 q = pq.PriorityQueue() # 定义优先队列, 默认第一元小根堆
24
25
26 def add_path(f, t, v): # 在前向星中加边
27     # 如果要修改全局变量, 要使用 global 来声明
28     global cnt, e, h
29     # 调试时的输出语句, 多个变量使用元组
30     # print("add_path(%d,%d,%d)" % (f,t,v))
31     cnt += 1
32     e[cnt].nex = h[f]
33     e[cnt].t = t
34     e[cnt].v = v
35     h[f] = cnt
36
37
38 def nextedgeid(u): # 生成器, 可以用在 for 循环里
39     i = h[u]
40     while i:
41         yield i
42         i = e[i].nex
43
44
45 def dijkstra(s):
46     dist[s] = 0
47     q.put((0, s))
48     while not q.empty():
49         u = q.get()
50         if dist[u[1]] < u[0]:
51             continue
52         for i in nextedgeid(u[1]):
53             v = e[i].t
54             w = e[i].v
55             if dist[v] <= dist[u[1]] + w:
56                 continue
57             dist[v] = dist[u[1]] + w
58             q.put((dist[v], v))
59
60
61 # 如果你直接运行这个 Python 代码 (不是模块调用什么的) 就执行命令
62 if __name__ == "__main__":
63     # 一行读入多个整数. 注意它会把整行都读进来
64     n, m, s = map(int, input().split())
65     for i in range(m):
66         u, v, w = map(int, input().split())
67         add_path(u, v, w)
68
69     dijkstra(s)
70
71     for i in range(1, n + 1):
72         # 两种输出语法都是可以用的
73         print("{}".format(dist[i]), end=" ")
74         # print("%d" % dist[i], end=' ')
75
76     print() # 结尾换行

```

## 参考文档

1. [Python Documentation](#)
2. [Python 官方中文教程](#)
3. [Learn Python3 In Y Minutes](#)
4. [Real Python Tutorials](#)
5. [廖雪峰的 Python 教程](#)
6. [GeeksforGeeks: Python Tutorials](#)

## 参考资料和注释

- 
1. [2. Python 解释器—Python 3 文档 ↗](#)
  2. [Unicode 指南—Python 3 文档 ↗](#)
- 

本页面最近更新：2025/2/2 13:28:46, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[cmpute](#), [Henry-ZHR](#), [ranwen](#), [abc1763613206](#), [billchenchina](#), [chinggg](#), [ChungZH](#), [CoelacanthusHex](#), [countercurrent-time](#), [Early0v0](#), [Entertainer](#), [F1shAndCat](#), [Great-designer](#), [hensier](#), [HeRaNO](#), [Hszzzx](#), [imba-tjd](#), [Ir1d](#), [jiangmuran](#), [ksyx](#), [lingxier](#), [LovelyBuggies](#), [Marcythm](#), [Mooos-MoSheng](#), [NachtgeistW](#), [ouuan](#), [Rottenwoood](#), [shawllewy](#), [shuzhouliu](#), [sshwy](#), [SukkaW](#), [Tiphereth-A](#), [tLLWtG](#), [wineee](#), [wxh06](#), [Xeonacid](#), [yusancky](#), [zyouxam](#), [zzjjbb](#), [Dong Tsing-hsuen](#), [Dong Tsing-hsuen](#), [mgt](#), [mgt](#), [Suyun514](#), [Suyun514](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用